Supplementary Material for "CascadeRAG: Abstention-Triggered Escalation that Outperforms Costlier Retrieval-Augmented Generation Architectures at a Fraction of their Cost"

Mikołaj Semeniuk
Agnieszka Jastrzębska

Faculty of Mathematics and Information Science, Warsaw University of Technology, Warsaw, Poland

S1 Background: how retrieval-augmented generation works

This section is a self-contained primer for readers new to dense retrieval and RAG; every concept introduced here is used in the article to explain a measured result.

S1.1 The knowledge problem, and why not simply retrain the model

A large language model stores what it knows in its weights (Brown et al., 2020; Petroni et al., 2019) — so-called *parametric* knowledge, acquired once, during training. Three consequences follow. First, the knowledge is *frozen*: a model trained in 2024 knows nothing that happened afterwards. Second, it is *incomplete and unreliable*: how much a model retains depends on how often it saw a fact (Kandpal, Deng, Roberts, Wallace, & Raffel, 2023; Roberts, Raffel, & Shazeer, 2020) — popular facts appear thousands of times where rare facts appear once or never (Mallen et al., 2023) — and when asked about the latter it tends to produce fluent, plausible, wrong answers — hallucinations (Ji et al., 2023; Zhang et al., 2023). Third, it is *closed*: an organisation's internal documentation, product catalogue, or case history was never in the training data at all.

The obvious remedy — continue training the model on the missing documents — is rarely practical. Full retraining of even a mid-sized model costs GPU-months; parameter-efficient fine-tuning (Hu et al., 2022) is cheaper but must be repeated every time the documents change, offers no attribution (the model still cannot say *where* an answer came from), and injects new factual knowledge less reliably than simply retrieving it (Ovadia, Brief, Mishaeli, & Elisha, 2023). Retrieval-Augmented Generation (Lewis et al., 2020) sidesteps the problem: the documents stay outside the model, in a searchable index that can be updated by inserting or deleting a record, and at question time the system retrieves the few passages most relevant to the question and prepends them to the model's prompt. The model's job shrinks from *recalling* the fact to *reading* it. Updating knowledge becomes a database operation rather than a training run; a company can place its private business context into the index without any of it entering the model's weights; and every answer can cite the passages it was read from.

S1.2 Embeddings and cosine similarity

The search step must match a question to passages by *meaning*, not just by shared words — "Where did the director of Beat Girl die?" shares almost no vocabulary with the encyclopedia paragraph that answers it. The standard tool is the *text embedding*: a neural network (the *embedding model* or *encoder*, typically a Transformer encoder (Devlin, Chang, Lee, & Toutanova, 2019; Vaswani et al., 2017)) that maps a piece of text to a vector of a few hundred real numbers,

trained so that texts with similar meaning land close together in that vector space (Reimers & Gurevych, 2019).

Closeness is measured by *cosine similarity*: the cosine of the angle between two vectors, which is 1 for parallel vectors, 0 for orthogonal ones, and -1 for opposite ones. For vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2} \sqrt{\sum_i b_i^2}}. \tag{1}$$

A toy example with three dimensions makes the mechanics concrete. Suppose an encoder mapped three texts as follows:

Text	Vector
"Who directed Beat Girl?"	(0.9, 0.4, 0.1)
"Beat Girl is a 1960 film directed by Edmond T. Gréville."	(0.8, 0.5, 0.2)
"The recipe calls for two eggs and a cup of flour."	(0.1, 0.2, 0.9)

The question and the film passage give $\cos = (0.72 + 0.20 + 0.02)/(1.0 \cdot 0.97) \approx 0.97$; the question and the recipe give $\cos \approx 0.35$. The film passage is retrieved. Real encoders behave the same way in 768 dimensions instead of three.

Two properties of embeddings matter for the results in this article. First, an embedding is a *lossy compression*: 768 numbers cannot preserve everything in a hundred-word passage, and what the compression loses first is rare surface forms — exact titles, names, codes. A question that names its target verbatim can therefore still miss it in embedding space, which is precisely the failure that lexical search repairs (Section 4.3 of the article). Second, some encoders are trained *asymmetrically*: the query is expected to carry a short instruction prefix (e.g. `query: ...`) that the passage side does not. Ignoring the convention silently degrades retrieval; we verify the convention for our encoder empirically (Section 6 of the article).

S1.3 Embedding models and cross-encoders

The encoder used for search is a *bi-encoder*: it embeds the question and each passage *separately*, and their relevance is the similarity of the two vectors. This is what makes web-scale search possible — all passage vectors are computed once, offline, and a question at query time is embedded once and compared against millions of stored vectors in milliseconds. The price is that the encoder never sees the question and the passage *together*, so it cannot model their interaction; it matches what a question is *about*.

A *cross-encoder* (Nogueira & Cho, 2019) reads the concatenated (question, passage) pair through one network and outputs a relevance score. Seeing both texts jointly, it can judge whether the passage actually answers the question — what the question is *asking for* — far more precisely, but at the cost of one full network pass per pair, which rules out scanning a corpus. The standard compromise is a two-stage pipeline: a bi-encoder retrieves a shortlist of candidates, and the cross-encoder reorders the shortlist (Section 4.4 of the article). The distinction between *about* and *asking for* is not cosmetic; it produces the single largest failure we measure (Section 7.2 of the article).

S1.4 Vector databases

Storing millions of embedding vectors and answering "which stored vectors are most similar to this query vector" is the job of a *vector database*. Comparing the query against every stored vector (exact search) is linear in corpus size; even GPU-accelerated exact search (Johnson, Douze, & Jégou, 2019) is a poor fit for serving interactive queries at the 21-million-passage scale used here. Vector databases therefore build an *approximate nearest neighbour* (ANN) index — most commonly a Hierarchical Navigable Small World graph (HNSW) (Malkov & Yashunin, 2020),

a layered proximity graph walked greedily from an entry point towards the query. The walk inspects a small fraction of the corpus and returns the nearest vectors it visited, trading a tunable amount of recall for orders of magnitude in speed; the size of the candidate list the walk keeps (`ef`) controls that trade, and we report and fix it in our setup because a benchmark that leaves it at an unknown default silently measures the index rather than the architecture.

Alongside each vector, the database stores the passage text and its identifier, so a search returns readable passages, ready to be placed in a prompt. The same database can also host a *sparse* (lexical) index, in which a passage is represented by weighted term counts and matching is by exact term overlap under BM25 scoring (Robertson & Zaragoza, 2009) — the modern packaging of classical keyword search, and the complement to dense retrieval used by the hybrid architectures below.

S2 Full per-set results

Recall is measured over the passages actually in the generator's context, whose mean size is reported beside it (Section 6.3 of the article); "EM answ." is Exact Match restricted to the questions the system did not decline. Dashes mark metrics that are undefined for a row (ClosedBook retrieves nothing; NaturalQuestions and TriviaQA annotate no gold documents, so answer-in-context replaces Recall there).

S2.1 MuSiQue

Architecture	EM	F1	Abst.	EM answ.	Recall	Passages	Calls
ClosedBook	0.0381	0.1186	0.1320	0.0439	–	–	1.00
BM25	0.0546	0.1162	0.3037	0.0784	0.2114	5.00	1.00
NaiveRAG-5	0.0596	0.1322	0.2607	0.0806	0.2209	5.00	1.00
NaiveRAG-10	0.0728	0.1532	0.2098	0.0921	0.2666	10.00	1.00
NaiveRAG-12	0.0712	0.1503	0.1816	0.0870	0.2812	12.00	1.00
Hybrid	0.0695	0.1462	0.2459	0.0922	0.2616	5.00	1.00
HyDE	0.0753	0.1596	0.2334	0.0983	0.2420	5.00	2.00
Rerank	0.0811	0.1634	0.2127	0.1030	0.2678	5.00	1.00
Neighbour	0.0691	0.1379	0.2205	0.0886	0.2209	12.26	1.00
CRAG-5	0.0832	0.1634	0.2315	0.1083	0.2326	5.35	2.99
CRAG-10	0.0877	0.1711	0.1953	0.1090	0.2732	10.00	3.00
IRCoT	0.0695	0.1478	0.2441	0.0920	0.2477	7.35	2.54
Adaptive	0.0666	0.1410	0.2482	0.0886	0.2365	6.64	2.96
Cascade	0.0840	0.1713	0.0335	0.0869	0.2391	4.41	1.58

S2.2 HotpotQA

Architecture	EM	F1	Abst.	EM answ.	Recall	Passages	Calls
ClosedBook	0.1887	0.2740	0.0204	0.1926	–	–	1.00
BM25	0.3063	0.4096	0.1164	0.3466	0.4810	5.00	1.00
NaiveRAG-5	0.3248	0.4313	0.1124	0.3659	0.5405	5.00	1.00
NaiveRAG-10	0.3299	0.4367	0.0905	0.3627	0.5994	10.00	1.00
NaiveRAG-12	0.3363	0.4448	0.0743	0.3632	0.6126	12.00	1.00
Hybrid	0.3368	0.4462	0.0912	0.3706	0.5646	5.00	1.00
HyDE	0.3338	0.4434	0.0974	0.3698	0.5552	5.00	2.00
Rerank	0.3757	0.4911	0.0791	0.4080	0.6408	5.00	1.00
Neighbour	0.3190	0.4251	0.0940	0.3521	0.5405	11.34	1.00
CRAG-5	0.3284	0.4367	0.1035	0.3663	0.5462	5.32	2.99
CRAG-10	0.3410	0.4505	0.0821	0.3715	0.6017	10.00	3.00
IRCoT	0.3409	0.4493	0.1030	0.3800	0.5629	6.13	2.31
Adaptive	0.3302	0.4378	0.1093	0.3707	0.5471	5.29	2.30
Cascade	0.3583	0.4713	0.0043	0.3598	0.5793	4.79	1.22

S2.3 2WikiMultihopQA

Architecture	EM	F1	Abst.	EM answ.	Recall	Passages	Calls
ClosedBook	0.2455	0.2969	0.0223	0.2511	–	–	1.00
BM25	0.2614	0.3222	0.1561	0.3097	0.3730	5.00	1.00
NaiveRAG-5	0.2285	0.2974	0.1672	0.2744	0.3540	5.00	1.00
NaiveRAG-10	0.2522	0.3196	0.1269	0.2888	0.3881	10.00	1.00
NaiveRAG-12	0.2516	0.3198	0.1172	0.2850	0.3960	12.00	1.00
Hybrid	0.2696	0.3338	0.1374	0.3125	0.4081	5.00	1.00
HyDE	0.2449	0.3132	0.1573	0.2906	0.3686	5.00	2.00
Rerank	0.2675	0.3311	0.1273	0.3065	0.3941	5.00	1.00
Neighbour	0.2395	0.3083	0.1483	0.2812	0.3542	10.72	1.00
CRAG-5	0.2332	0.3014	0.1808	0.2846	0.3375	5.17	3.00
CRAG-10	0.2535	0.3205	0.1350	0.2930	0.3758	10.00	3.00
IRCoT	0.2467	0.3165	0.1556	0.2921	0.3887	6.96	2.48
Adaptive	0.2289	0.2978	0.1678	0.2750	0.3560	5.14	2.11
Cascade	0.2783	0.3436	0.0052	0.2797	0.3808	4.67	1.32

S2.4 NaturalQuestions

Architecture	EM	F1	Abst.	EM answ.	Ans. in ctx	Passages	Calls
ClosedBook	0.1399	0.2374	0.0823	0.1524	–	–	1.00
BM25	0.2249	0.3166	0.2349	0.2940	0.4559	5.00	1.00
NaiveRAG-5	0.3349	0.4436	0.1693	0.4031	0.7262	5.00	1.00
Hybrid	0.3089	0.4228	0.1748	0.3743	0.6707	5.00	1.00
HyDE	0.3476	0.4602	0.1598	0.4138	0.7525	5.00	2.00
Rerank	0.2352	0.3177	0.2773	0.3254	0.5097	5.00	1.00
CRAG-5	0.3415	0.4529	0.1660	0.4094	0.7554	5.53	2.94
IRCoT	0.3357	0.4451	0.1657	0.4024	0.7298	5.29	2.09
Adaptive	0.3352	0.4433	0.1704	0.4040	0.7253	5.01	2.05
Cascade	0.3417	0.4603	0.0316	0.3528	0.6748	4.52	1.44

S2.5 TriviaQA

Architecture	EM	F1	Abst.	EM answ.	Ans. in ctx	Passages	Calls
ClosedBook	0.4241	0.4975	0.0147	0.4304	–	–	1.00
BM25	0.5356	0.6220	0.0994	0.5947	0.6706	5.00	1.00
NaiveRAG-5	0.5700	0.6594	0.0796	0.6193	0.7358	5.00	1.00
Hybrid	0.5835	0.6732	0.0666	0.6252	0.7494	5.00	1.00
HyDE	0.5954	0.6854	0.0665	0.6378	0.7681	5.00	2.00
Rerank	0.5962	0.6847	0.0631	0.6364	0.7625	5.00	1.00
CRAG-5	0.5845	0.6739	0.0720	0.6298	0.7560	5.39	2.97
IRCoT	0.5723	0.6623	0.0756	0.6190	0.7416	5.38	2.11
Adaptive	0.5698	0.6593	0.0791	0.6187	0.7346	5.02	2.06
Cascade	0.6032	0.6933	0.0044	0.6059	0.7617	4.84	1.16

S2.6 Cascade ablations (Exact Match)

Variant	MuSiQue	HotpotQA	2Wiki	NQ	TriviaQA
Stage 1 alone (fused, no escalation)	0.0783	0.3521	0.2630	0.3257	0.5921
Cascade (proposed)	0.0840	0.3583	0.2783	0.3417	0.6032
Cascade, rerank first	0.0840	0.3825	0.2823	0.2951	0.6079
Cascade, naive first	0.0662	0.3353	0.2542	0.3474	0.5869
Cascade, hybrid first	0.0757	0.3456	0.2819	0.3269	0.5973
Cascade + confidence trigger	0.0841	0.3562	0.2745	0.3408	0.6053
IRCoT, one-shot	0.0712	0.3367	0.2565	0.3358	0.5756

S2.7 Second reader (Llama-3.1-8B-Instruct, Exact Match)

Architecture	MuSiQue	HotpotQA	2Wiki	NQ	TriviaQA
ClosedBook	0.0451	0.2182	0.2932	0.2463	0.5839
NaiveRAG-5	0.0728	0.3355	0.2228	0.3493	0.5885
Rerank	0.1031	0.3870	0.2603	0.2617	0.6100
Stage 1 alone (fused)	0.0869	0.3633	0.2550	0.3572	0.6086
Cascade	0.0961	0.3704	0.2770	0.3658	0.6168
Cascade, calls/question	1.41	1.14	1.29	1.16	1.08

Baseline (calls/q)	NQ	TriviaQA	HotpotQA	2Wiki	MuSiQue
ClosedBook (no retrieval) (1.0)	**+0.119**	**+0.033**	**+0.152**	-0.016	**+0.051**
NaiveRAG, 5 passages (1.0)	**+0.016**	**+0.028**	**+0.035**	**+0.054**	**+0.023**
Rerank (cross-encoder) (1.0)	**+0.104**	+0.007	-0.016	**+0.017**	-0.007
Fused retrieval (stage 1 alone) (1.0)	**+0.009**	**+0.008**	**+0.007**	**+0.022**	**+0.009**
Cascade calls/q	1.16	1.08	1.14	1.29	1.41

S3 The confidence-threshold sweep

Swept offline over 4988 sampled train questions by composing the three independently-run stages question by question (Section 5.3 of the article). "Escalated" counts stage-1 answers handed on at that threshold, "discarded" the correct answers among them, "missed" those no later stage recovered. Exact Match peaks at 0.3941 (threshold -0.150) against 0.3923 for the abstention-only cascade.

Threshold	EM	F1	Calls	Escalated	Discarded	Missed
$-\infty$ (abstention only)	0.3923	0.4708	1.35	687	0	0
-2.000	0.3923	0.4708	1.35	687	0	0
-1.500	0.3923	0.4708	1.35	688	0	0
-1.000	0.3923	0.4708	1.35	697	2	1
-0.800	0.3923	0.4707	1.37	723	7	2
-0.600	0.3921	0.4701	1.38	755	16	3
-0.500	0.3925	0.4706	1.40	785	26	6
-0.400	0.3937	0.4714	1.42	831	33	8
-0.300	0.3939	0.4717	1.46	909	50	12
-0.200	0.3917	0.4701	1.53	1046	93	33
-0.150	0.3941	0.4714	1.60	1169	115	41
-0.100	0.3941	0.4722	1.72	1385	159	62
-0.050	0.3939	0.4701	1.94	1764	216	93
-0.020	0.3919	0.4662	2.20	2194	312	139
-0.010	0.3917	0.4653	2.36	2468	378	167
-0.005	0.3865	0.4601	2.50	2691	464	208

S4 Prompts

All prompts, verbatim from the implementation. The reader prompt is shared by every architecture; the instruction to output only the answer exists because the gold answers are short spans (Section 6.3 of the article).

Reader (all architectures).

```
Answer the question based on the given documents.  Only give me the answer
and do not output any other words.
Documents:
[1] ... [k] ...
Question:  {question}
```

ClosedBook reader. As above, minus the documents block.

HyDE drafter.

```
Write a short encyclopedia passage that answers the question.
Question:  {question}
Name the people, places, dates and titles involved.  Write it as an
encyclopedia entry, not as a reply to the question.  If you are not sure of
the facts, write the most plausible passage anyway.  Reply with only the
passage.
```

IRCoT reasoning step.

```
You are answering a question step by step using the documents below.
Documents:  ...
Question:  {question}
Reasoning so far:  ...
Write the next single sentence of reasoning.  Name the entities that still
need to be looked up.  If the documents already answer the question, write
"So the answer is:  <answer>".
```

The one-shot variant prepends a single hand-written two-hop worked example (identical for every question set).

CRAG retrieval grader.

```
Judge which documents contain information useful for answering the question.
Documents: ...
Question: {question}
Reply with only the numbers of the useful documents, separated by commas.
If none of them are useful, reply with "none".
```

CRAG query rewrite.

```
The search results for this question were not useful.
Question: {question}
Write a better search query for finding the documents that answer it. Spell
out the entities the question only refers to indirectly. Reply with only
the query.
```

Adaptive-RAG router.

```
Classify how much looking up the question needs.
Question: {question}
A - a widely known fact that needs no source, such as a famous person,
country or date.
B - one fact that has to be looked up in a single document.
C - several facts from different documents that have to be combined, for
example when the question refers to an entity it does not name.
Reply with only the letter A, B or C.
```

Abstention phrases. An answer is counted as an abstention if, after normalisation, it contains any of: *do(es) not contain, no information, not mentioned, can(not) answer, not provided, no mention, unable to answer, not enough information, insufficient information, there is no, not possible to determine* — or if it is longer than twenty words.

References

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12).

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*(3).

Kandpal, N., Deng, H., Roberts, A., Wallace, E., & Raffel, C. (2023). Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *42*(4).

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL).*

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Ovadia, O., Brief, M., Mishaeli, M., & Elisha, O. (2023). Fine-tuning or retrieval? comparing knowledge injection in LLMs. *arXiv preprint arXiv:2312.05934*.

Petroni, F., Rocktäschel, T., Lewis, P., Bakhtin, A., Wu, Y., Miller, A. H., & Riedel, S. (2019). Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

Roberts, A., Raffel, C., & Shazeer, N. (2020). How much knowledge can you pack into the parameters of a language model? In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4).

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30).

Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., ... Shi, S. (2023). Siren's song in the AI ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*.